

Teoría de la Computabilidad

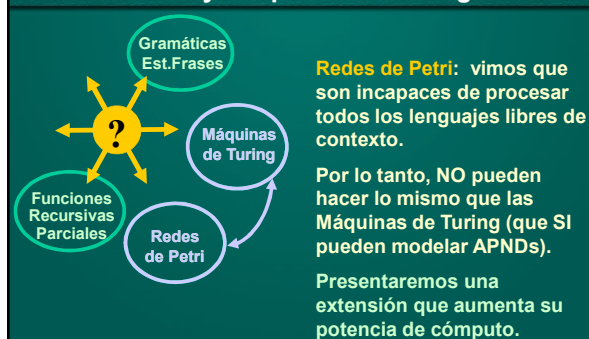
Módulo 18: Teoremas de Equivalencia Redes de Petri y Máquinas de Turing

Departamento de Cs. e Ing. de la Computación
Universidad Nacional del Sur
Bahía Blanca, Argentina

Relación entre Modelos Estudiados



Redes de Petri y Máquinas de Turing



RP: Incorporando Testeo por Cero

Capacidad de testeo por cero: habilidad de una RP para verificar que en un determinado lugar no hay tokens (o equivalentemente, hay cero tokens).

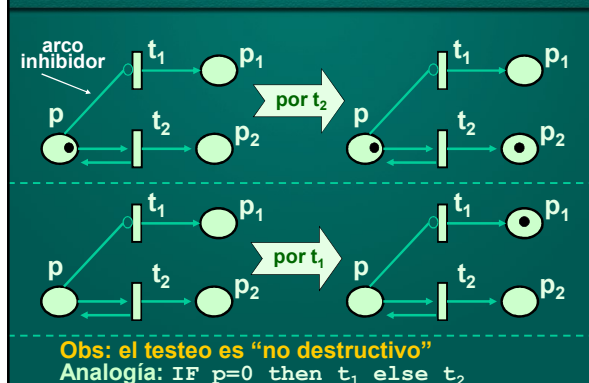
La idea es considerar dos tipos de arcos: *comunes* e *inhibidores*.

Arco común: permite que una transición sea disparada siempre que en el nodo desde el cual parte el arco haya un token para “alimentarlo”.

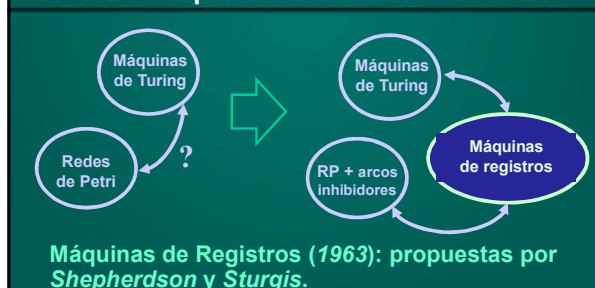
Arco inhibidor: permite que una transición se dispare si en el nodo desde el cual parte no hay tokens.



Arcos comunes vs. inhibidores



Estudiando la potencia de las RP



Máquina de Registros (MR)

Una MR es un dispositivo de cómputo que asume la existencia de un nro. finito de unidades de almacenamiento (registros) y un programa formado por instrucciones secuenciales.

Registro: guarda nro. de longitud arbitraria.

Repertorio de instrucciones:

- **Inc(R_n):** incrementa R_n en uno.
- **Dec(R_n):** decrementa R_n en uno.
- **Saltar(R_n)[i]:** salta a la instrucción i si el registro R_n contiene el valor 0.
- **Parar:** detiene la ejecución del programa.

Ejemplo: Máquina de Registros

Sumar el contenido del registro R_2 al registro R_1 .

Para esto puede suponerse que R_0 contiene inicialmente 0 (=salto incondicional).

Un programa que resuelve el problema es el siguiente...

- 1) Saltar(R_2)[5]
- 2) Dec(R_2)
- 3) Inc(R_1)
- 4) Saltar(R_0)[1]
- 5) Parar

$R_1 = 5 \Rightarrow R_1 = 7$
 $R_2 = 2 \Rightarrow R_2 = 0$

Máquina de Registros

10	Inc(R_1)
11	Dec(R_2)
12	Saltar(R_2)[14]
13	Saltar(R_0)[10]
14	Parar
15	

La memoria de la computadora puede pensarse como un **arreglo**. Cada celda tiene una dirección que la identifica.

Cuando la memoria almacena un programa de la MR, cada celda guarda una instrucción.

Salto absolutos vs. relativos

10	Inc(R_1)
11	Dec(R_2)
12	Saltar(R_2)[14]
13	Saltar(R_0)[10]
14	Parar
15	

Salto absolutos

dirección absoluta

10	Inc(R_1)
11	Dec(R_2)
12	Saltar(R_2)[+2]
13	Saltar(R_0)[-3]
14	Parar
15	

Salto relativos

desplazamiento

Problema de Saltos absolutos

10	Inc(R_1)	23	Inc(R_1)
11	Dec(R_2)	24	Dec(R_2)
12	Saltar(R_2)[14]	25	Saltar(R_2)[14]
13	Saltar(R_0)[10]	26	Saltar(R_0)[10]
14	Parar	27	Parar
15		28	

Si hay algún salto absoluto, ¡un programa no se puede cambiar de lugar de memoria!

Si todo salto es relativo, no hay problema

Máquina de Registros

10	Inc(R_1)
11	Dec(R_2)
12	Saltar(R_2)[14]
13	Saltar(R_0)[10]
14	Parar
15	

¿Cómo sabe la computadora cuál instrucción está ejecutando en cada momento?

Usa un registro especial llamado “**program counter**” (o PC).

7 1 0 10
 R_1 R_2 R_0 PC

Ejecutando Saltar absoluto

10	Inc(R ₁)
11	Dec(R ₂)
12	Saltar(R ₂)[14]
13	Saltar(R ₀)[10]
14	Parar
15	

¿Cómo hace la computadora para ejecutar un **Saltar** absoluto?

Cambia el valor del "program counter".

8	0	0	12
R ₁	R ₂	R ₀	PC

Salto absoluto = se asigna 14 al PC

14	PC
----	----

Ejecutando Saltar relativo

10	Inc(R ₁)
11	Dec(R ₂)
12	Saltar(R ₂)[+2]
13	Saltar(R ₀)[-3]
14	Parar
15	

¿Cómo hace la computadora para ejecutar un **Saltar** relativo?

Cambia el "program counter" según desplazam.

8	0	0	12
R ₁	R ₂	R ₀	PC

PC ← PC + 2

14	PC
----	----

Lema: MR → RP

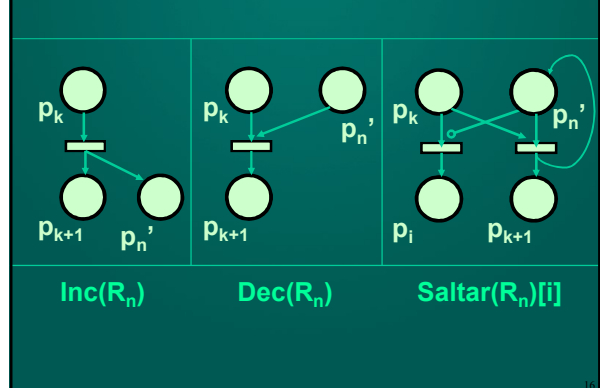
Lema: Todo problema que puede resolverse con máquinas de registros puede resolverse con una red de Petri.

Demostración: Debemos mostrar que toda instrucción de una máquina de registros se puede expresar a través de una red de Petri.

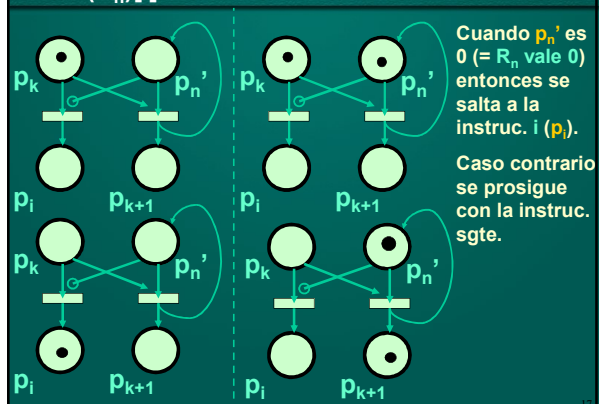
registros = **lugares** en una RP.

instrucciones = **transiciones** en una RP.

Instrucciones con RP



Saltar(R_n)[i]

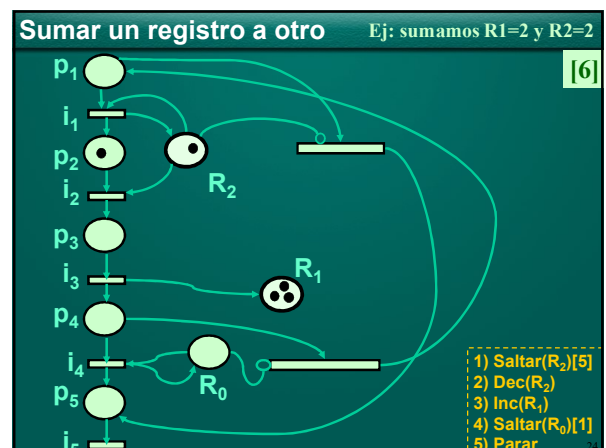
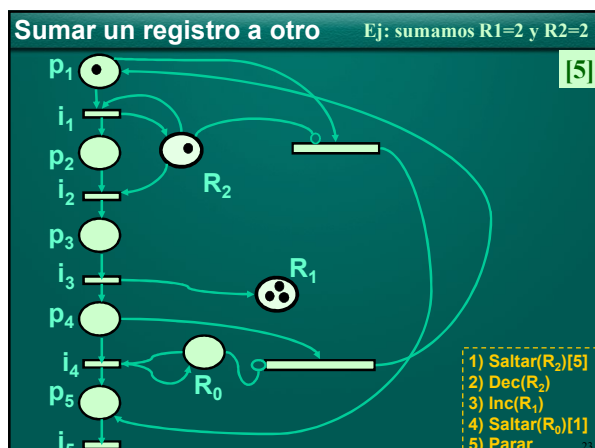
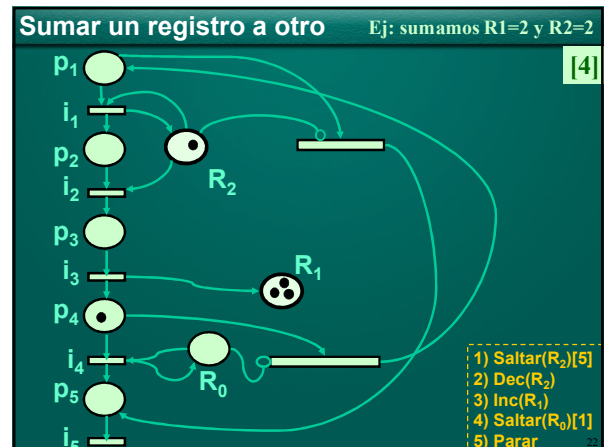
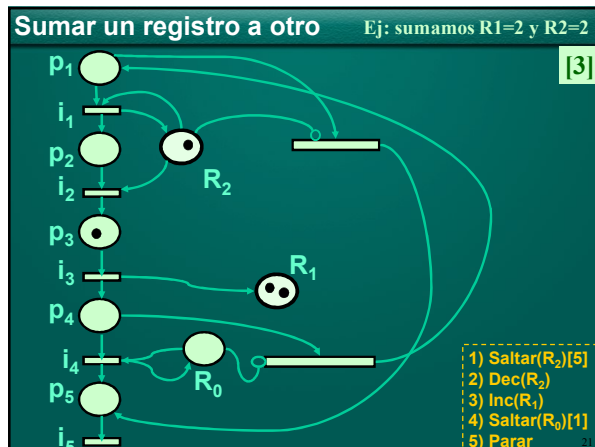
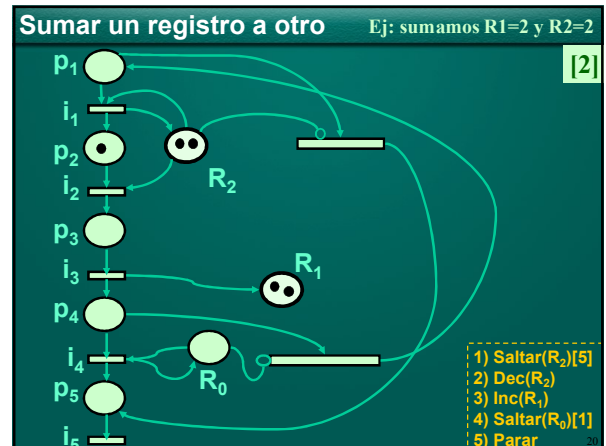
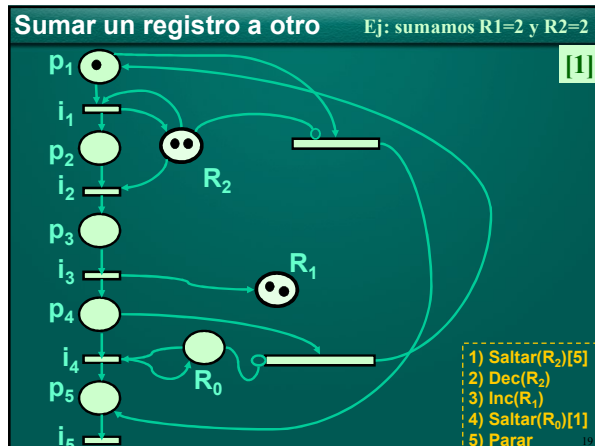


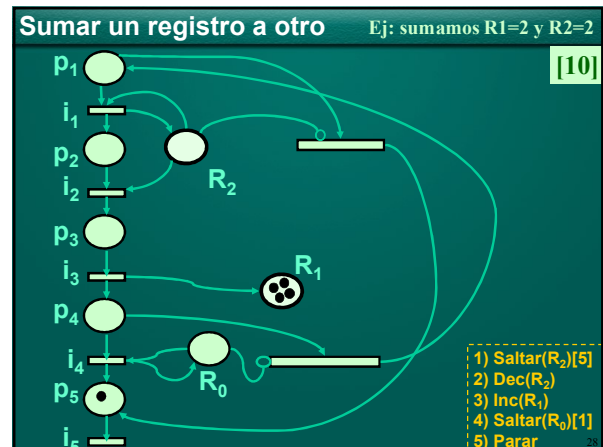
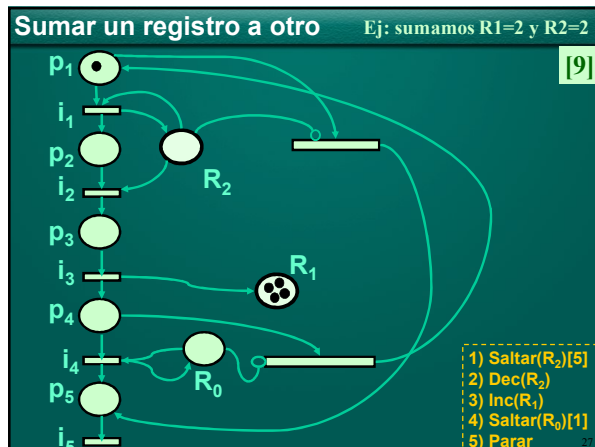
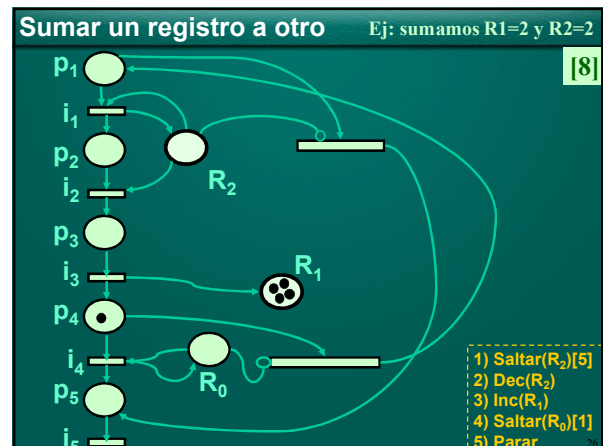
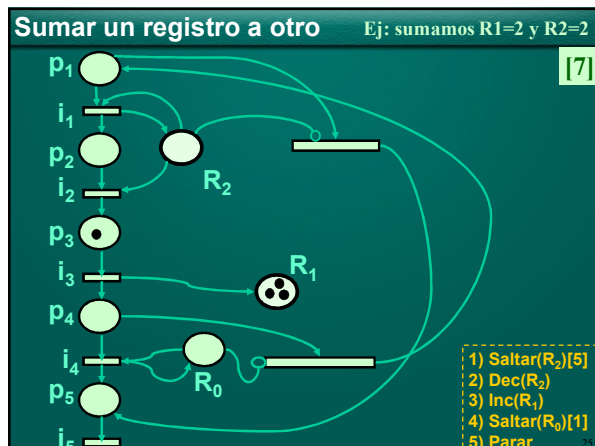
Ejemplo: Suma de dos registros

Consideremos dos registros R_2 y R_1 , tal que queremos calcular $R_1 \leftarrow R_1 + R_2$.

Ubicamos c y c' tokens en R_1 y R_2 , respectivamente. Ubicamos un token en p_1 (flujo de la información en el programa).

Queremos armar una RP tal que se llegue a 0 tokens en R_2 y $c+c'$ en R_1 (puede usarse para esto el programa visto, codificando sus instrucciones con RPs).





Lema: $RP \rightarrow MR$

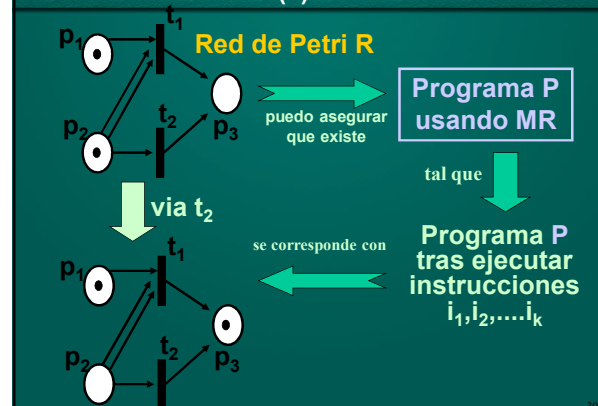
Lema: Todo problema que puede resolverse con una red de Petri es soluble a través de una máquina de registros.

Demostración [Esquema]: Debemos mostrar cómo simular la dinámica de una RP, representando lugares y transiciones.

Registro = un lugar en la RP

Transición disparada = involucrará un ciclo en MR que decrementará e incrementará los registros correspondientes.

RP $\rightarrow$ MR / Intuición (2)



Enfoque para probar RP $\rightarrow$ MR (1)

Lugares = registros

Lugar con k tokens = un registro con valor k .Sea $R=(P,T,IF,OF)$. Supongamos $|P|=n$ y $|T|=m$.Tendremos n registros $r_1, r_2, \dots, r_n$ para representar los lugares de R .Para las transiciones, dispondremos de $2nm$ registros, destinados a registrar arcos incidentes (p_i, t_j) y arcos salientes (t_j, p_i) .

El valor de estos registros no cambia (pues representan fcs. IF y OF que caracterizan una RP).

31

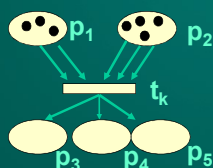
Enfoque para probar RP $\rightarrow$ MR (2)Notación: if_{ij} = registro cuyo contenido es $IF(p_i, t_j)$ of_{ij} = registro cuyo contenido es $OF(t_j, p_i)$

Un programa que simule una RP contendrá dos bloques de instrucciones principales:

- Inicialización
- Disparo de transiciones (*traslado de tokens*)

32

Una RP en términos de registros



r_1	3	if_{1k}	2
r_2	4	if_{2k}	3
r_3	0	of_{k3}	1
r_4	0	of_{k4}	1
r_5	0	of_{k5}	1

33

Esquema RP $\rightarrow$ MR (3)

Inicialización

Trasladar Tokens

Retirar
Tokens
Depositar
Tokens

Inicialización: instrucciones que colocan en el cijo. $r_1, r_2, \dots, r_n$ los números que representan el contenido de $p_1, \dots, p_n$. También se definen los registros if_{ij} y of_{ij} . Se asumirá también $r_0 = 0$. Eventualmente habrá otros registros auxiliares inicializados en 0.

34

Esquema RP $\rightarrow$ MR (4)

Inicialización

Trasladar Tokens

Retirar
Tokens
Depositar
Tokens

Testeo y Disparo de transiciones: para cada trans. t_k destinar un trozo de programa que duplique el contenido de $r_1, r_2, \dots, r_n, if_{11}, \dots, if_{nm}, of_{11}, \dots, of_{mn}$ en los registros aux. $r'_1, r'_2, \dots, r'_n, if'_{11}, \dots, if'_{nm}, of'_{11}, \dots, of'_{mn}$.

Veamos cómo hacerlo...

35

Copiando registros

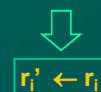
s-1: Saltar(r_i)[s+9]s : Dec(r_i)s+1: Inc(r'_i)s+2: Inc(a_i')s+3: Saltar(r_i)[s+5]s+4: Saltar(r_0)[s]s+5: Inc(r_i)s+6: Dec(a_i')[s+9]s+7: Saltar(a_i')[s+9]s+8: Saltar(r_0)[s+5]

s+9:

Situación inicial: r_i tiene un valor x , r'_i y a_i' están en 0

Situación final: r_i tiene valor x , r'_i tiene valor x .

Nota: el registro a_i' es usado para almacenar auxiliar.



36

Ejercicio propuesto

Se deja como ejercicio mostrar que con las instrucciones de máquinas de registros pueden escribirse el **condicional** y la **repetición condicionada** de Pascal.

En lo que sigue usaremos estas estructuras de control en lugar de MR puras, para simplificar su comprensión.

37

Cómo averiguar si t_k está habilitada

Para $i = 1 \dots n$:

mientras $if'_{ik} > 0$

Dec(r'_i)

Dec(if'_{ik})

Habilitada = $r'_1 \geq 0 \wedge \dots \wedge r'_n \geq 0$

(se decrementó cada if'_{ik} tal que $IF(p_i, t_j) > 0$)

Si la red tuviera algún arco inhibitor desde un lugar p_i hacia t_k , el último test debería contemplar esto verificando que $M(p_i) = 0$.

38

Efecto de disparar una transición t_k

Si **Habilitada** entonces:

Duplicar $if_{1k}, \dots, if_{nk}, of_{k1}, \dots, of_{km}$

en $if'_{1k}, \dots, if'_{nk}, of'_{k1}, \dots, of'_{km}$

Para $i = 1 \dots n$: {para c/arco entrada a t_k }

Mientras $if'_{ik} > 0$

Dec(r'_i)

Dec(if'_{ik})

Para $i = 1 \dots m$: {para c/arco salida desde t_k }

Mientras $of'_{ki} > 0$

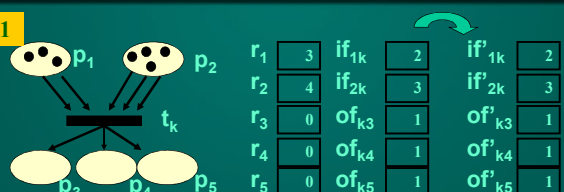
Inc(r'_i)

Dec(of'_{ki})

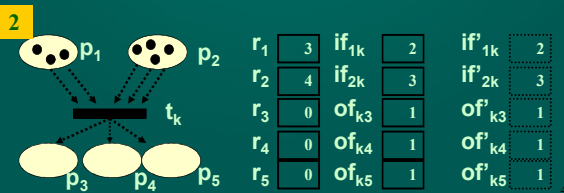
39

Efecto de disparar una transición t_k

1



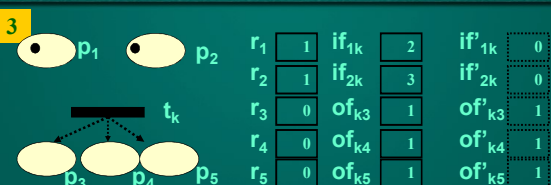
2



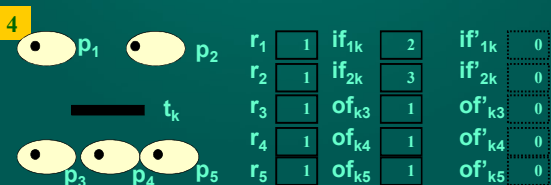
40

Efecto de disparar una transición t_k

3



4



41

Teorema de Equivalencia MR $\leftrightarrow$ RP

Teorema: Las máquinas de registros son equivalentes en poder computacional a las Redes de Petri.

Demostración: Se deduce de los dos lemas anteriores.

42

Lema: M. de Turing --> MR

Lema: Las máquinas de Turing pueden simular las máquinas de registros.

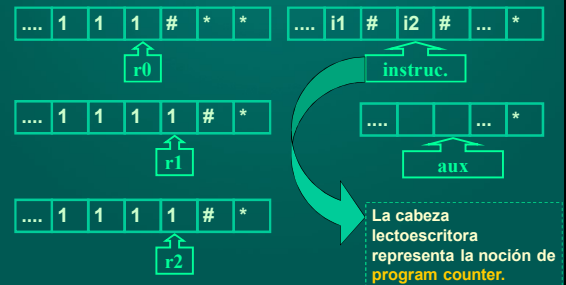
Demostración: Supongamos una MR R con k registros. Asumiremos una MT de $k+2$ cintas, donde

- c/u de las primeras k cintas representa un registro de R .
- la cinta $k+1$ -ésima almacena la secuencia de instrucciones a simular.
- la cinta $k+2$ -ésima se usa para cálculos auxiliares.

Nota: supondremos que T usa notación unaria.

Intuición: MR representada con MT

Supongamos una MR R con 3 regs. r_0, r_1 , y r_2 .

**Lema: M. de Turing --> MR (1)**

Para simular la instrucción $\text{Inc}(R_n)$ mediante T basta con agregar un número 1 al final de la cadena de la cinta n -ésima.

Análogamente puede implementarse $\text{Dec}(R_n)$: basta con quitar un número 1 al final de la cadena de la cinta n -ésima.

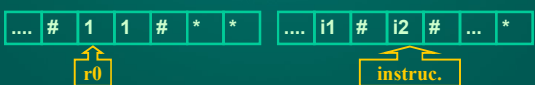
**Lema: M. de Turing --> MR (2)**

Veamos ahora cómo simular $\text{Saltar}(R_n)[i]$. Se requieren varias operaciones en T :

- 1) Verificar si la cinta n -ésima contiene la cadena $\#1\#$, la cual representaría que el registro n -ésimo tiene como contenido el nro. 0.
- 2) Si la cinta n -ésima NO contiene el nro. 0 en notación unaria, avanzar la cabeza lectoescritora (*program counter*) de la cinta $k+1$ para pasar a la instrucción siguiente.

Ejemplo

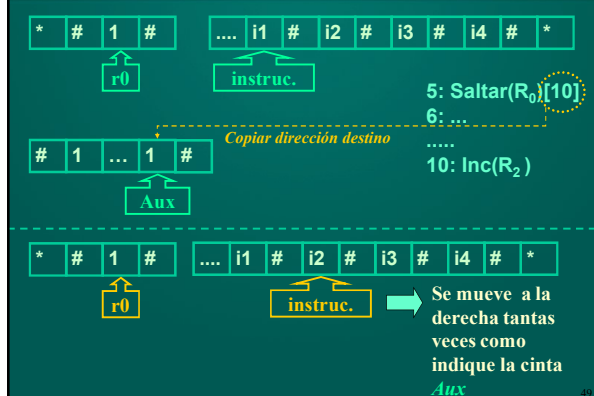
Tras ejecutar $\text{Saltar}(R_0)[i]$

**Lema: M. de Turing --> MR (3)**

Pasos restantes para simular $\text{Saltar}(R_n)[i]$:

- 3) Si la cinta n -ésima CONTIENE el nro. 0 en notación unaria, deberá avanzarse y saltar hasta un cierto número de instrucción i .
 - a) Copiar i en la cinta $k+2$; decrementarlo dos veces.
 - b) Llevar la cabeza de la cinta $k+1$ al extremo izq. (primera instrucción de la MR simulada).
 - c) A menos que la cabeza en cinta $k+2$ esté leyendo $\#$, decrementar cinta $k+2$ y avanzar en cinta $k+1$.

Ejemplo: ejecución de $\text{Saltar}(R_0)[i]$



Lema: M. de Turing $\rightarrow$ MR (4)

Veamos ahora cómo simular **Parar**. Esto se simula logrando la detención de la máquina de Turing.

Es bastante simple. En este caso, debemos dejar a la MT en un estado aceptador y utilizar la opción N (para indicar que las cabezas lecto escritoras no se deben mover).

Lema: MR $\rightarrow$ M.Turing (1)

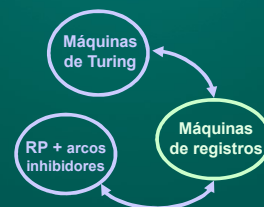
Lema: Las máquinas de registros pueden simular a las máquinas de Turing.

(No presentaremos la demostración de esta segunda parte del Lema)

Conclusiones

Teo: Las MR son equivalentes en poder computacional a las máquinas de Turing.

Dem : Se deduce de los lemas antes vistos.



Síntesis

